HDOC Day-46 Activity

Variable-Length Array, Bubble Sort, Pointers and Occurrence Search

Name	Gurveer Singh Cheema
SAF ID	590032925
Batch	15

Aim

To read a variable-length integer array, pass it between functions using pointers, sort it in ascending order using Bubble Sort, and find the first and last occurrence of a target value in the sorted array.

Requirements

1. Read the size and elements of **nums[]**.
2. Pass the array by address to **Bubble_sort()** and sort it in ascending order.
3. Pass the sorted array to **occurrence()** using a pointer.
4. Read a target value.
5. Display the first and last indices of the target.
6. If the target is absent, display **-1, -1**.

Algorithm

Bubble_sort():
1. Compare adjacent elements.
2. If the left element is greater than the right element, swap them.
3. Repeat for all elements.
4. After each pass, the largest unsorted element reaches its correct position.
5. Continue until the array is sorted.

occurrence():
1. Read target.
2. Initialize first = -1 and last = -1.
3. Scan the sorted array from index 0 to n-1.
4. On the first match, store the index in first.
5. For every match, update last.
6. If first remains -1, print -1, -1; otherwise display the value and both indices.

C Program

```
#include <stdio.h>

void Bubble_sort(int *nums, int size)
{
    int i, j, temp;

    for (i = 0; i < size - 1; i++)
    {
        for (j = 0; j < size - i - 1; j++)
        {
            if (*(nums + j) > *(nums + j + 1))
            {
                temp = *(nums + j);
                *(nums + j) = *(nums + j + 1);
```

```
                *(nums + j + 1) = temp;
            }
        }
    }
}

void occurrence(int *nums, int size)
{
    int target, i;
    int first = -1, last = -1;

    printf("Enter target: ");
    scanf("%d", &target);

    for (i = 0; i < size; i++)
    {
        if (*(nums + i) == target)
        {
            if (first == -1)
                first = i;

            last = i;
        }
    }

    if (first == -1)
    {
        printf("-1, -1\n");
    }
    else
    {
        printf("First occurrence: %d at index %d\n",
               target, first);
        printf("Last occurrence: %d at index %d\n",
               target, last);
    }
}

int main()
{
    int size, i;

    printf("Enter size of array: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("Invalid array size!\n");
        return 0;
    }

    int nums[size];

    printf("Enter %d elements:\n", size);

    for (i = 0; i < size; i++)
        scanf("%d", &nums[i]);

    Bubble_sort(nums, size);

    printf("Sorted array: ");

    for (i = 0; i < size; i++)
        printf("%d ", nums[i]);

    printf("\n");

    occurrence(nums, size);

    return 0;
}
```

Sample Outputs

```
Example 1
Enter size of array: 6
Enter 6 elements:
5 7 7 8 8 10
Sorted array: 5 7 7 8 8 10
Enter target: 8
First occurrence: 8 at index 3
Last occurrence: 8 at index 4

Example 2
Enter size of array: 5
Enter 5 elements:
5 7 7 8 10
Sorted array: 5 7 7 8 10
Enter target: 8
First occurrence: 8 at index 3
Last occurrence: 8 at index 3

Example 3
Enter size of array: 6
Enter 6 elements:
5 7 7 7 8 10
Sorted array: 5 7 7 7 8 10
Enter target: 6
-1, -1
```

Function Explanation

Bubble_sort(int *nums, int size): The array address is received in the pointer nums. Pointer expressions such as ***(nums + j)** access array elements. Adjacent elements are swapped whenever they are in the wrong order.

occurrence(int *nums, int size): Reads the target and scans the sorted array. The first matching index is stored only once, while last is updated for every match. If there is no match, both values remain -1.

Pointer Concept Used

When an array is passed to a function, its base address is passed. Therefore **nums[i]** and ***(nums + i)** refer to the same array element. Changes made by Bubble_sort() directly modify the original array.

Easy Memory Tricks

Bubble Sort: Compare -> Swap -> Largest moves to the end.
First occurrence: Set only when first == -1.
Last occurrence: Update on every match.
Not found: first = -1 and last = -1.

Complexity

Bubble Sort: O(n^2) time in this implementation.
Occurrence search: O(n) time.
Extra space: O(1), excluding the input variable-length array.

Result

Thus, the variable-length array was passed using pointers, sorted in ascending order using Bubble Sort, and the first and last occurrences of the target were found successfully.

Quick Viva Questions

1. What is a VLA? A variable-length array whose size is decided at run time.
2. What does *(nums + i) mean? The element at index i of the array.
3. Why are pointers used? They pass the array address to functions so the original array can be

accessed and modified.

4. What is Bubble Sort? A sorting method that repeatedly compares and swaps adjacent elements.

5. What is printed if target is absent? -1, -1.

6. If the target occurs once? First and last occurrence indices are the same.